Rigorous Evaluation of LLM Components in HPC Research

Section I: Introduction

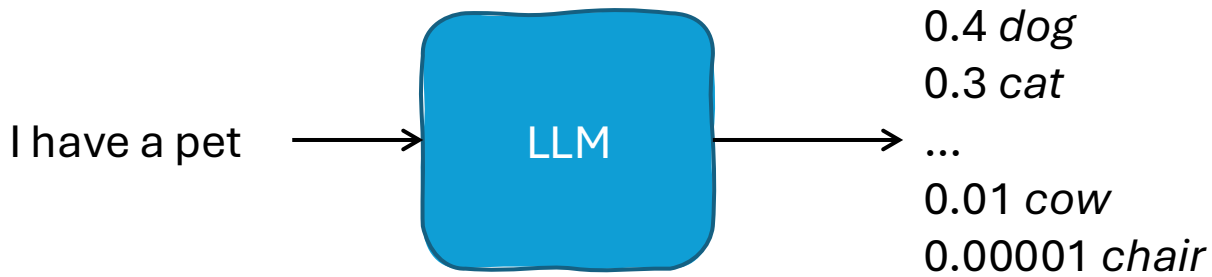
- Tutorial Objectives
  - Learn how to properly evaluate LLMs in HPC and scientific research
  - Learn the dangers and common pitfalls from using LLMs in research
- Overview
  - Intro to LLMs
  - How to Lie with LLM Results
  - Experimental Design for Stochastic Components
  - Transparency and Reproducibility
  - Human Validation: When and How
  - Baselines, Metrics, and Ablations
  - HPC and Scientific Computing Specifics

Section II: Introduction to LLMs

- Goal for this section: shared vocabulary for using LLMs
- You should be able to answer:
  - What are tokens?
  - What is text generation?
  - Where does randomness come from?
  - What is a model?

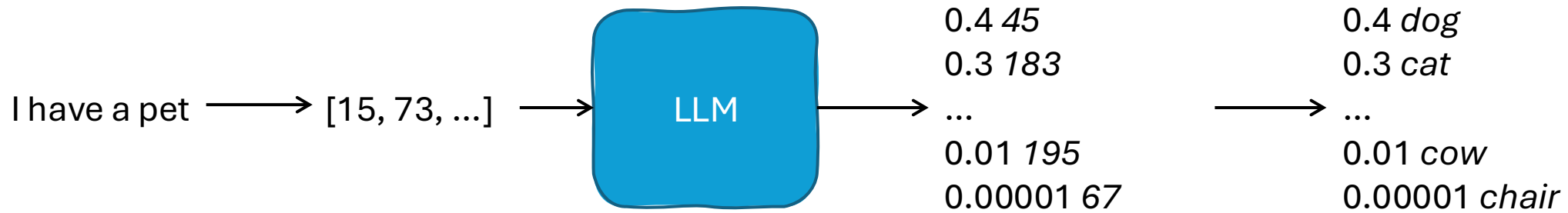
What is an LLM?

- Large Language Models are neural networks based on the transformer architecture that are state-of-the-art for modeling sequential data (e.g. text, language)
- Often trained for causal language modeling
  - Input: text sequence → output: probabilities of next token



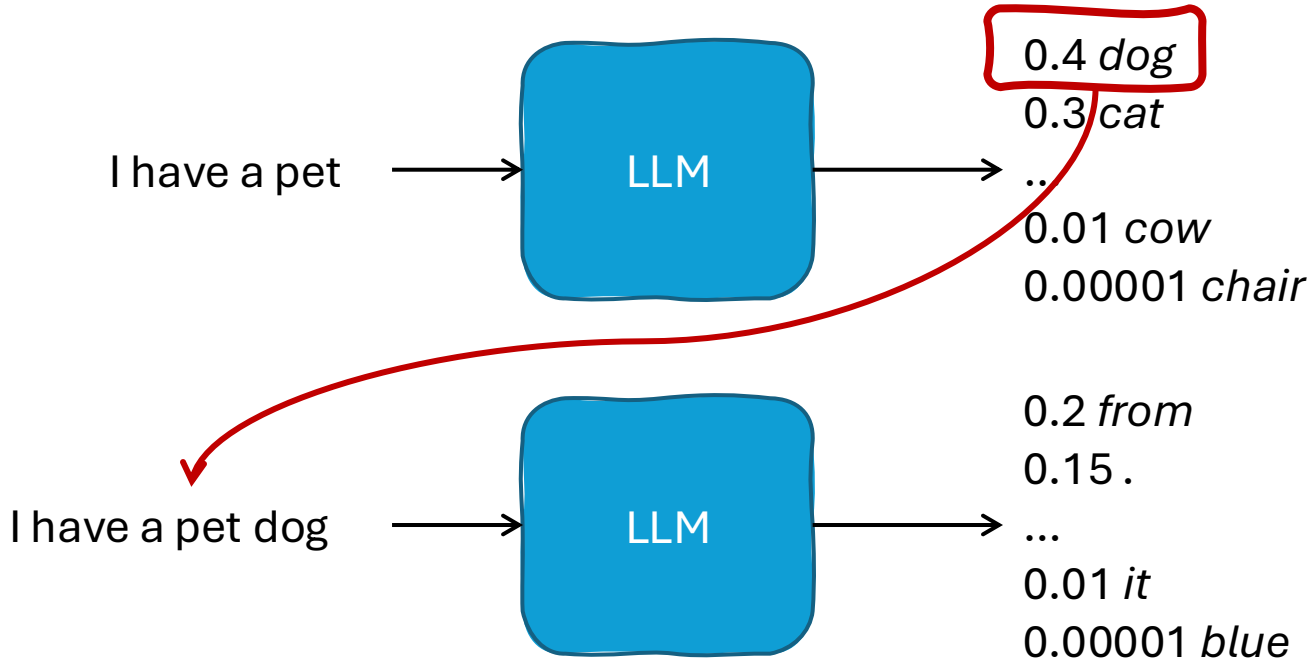
What are tokens?

- LLMs learn from integer sequences, not text
- *Tokenization* is the process of mapping text to integer sequences
- LLMs take sequences of tokens as input and outputs token probabilities
- Tokens *do not* correspond 1-1 with words or characters



Text Generation

- Auto-regressive text generation is the process of repeatedly sampling the next token to generate sequences



Text Generation

- Auto-regressive text generation is the process of repeatedly sampling the next token to generate sequences
- Stop when the model generates stop or EOS token
- Note: early differences in sequences can cascade into very different outputs

Sampling: The Introduction of Randomness

- Using the most probable next token leads to bad, repetitive generations (A. Holtzman et al. “The Curious Case of Neural Text Degeneration”)
- Randomly sampling yields much better results
- Popular sampling strategies
  - Top-K
  - Top-P or Nucleus Sampling
  - Beam search
- Temperature often used to scale probabilities before sampling

Randomness in LLMs

- Transformers with fixed weights and inputs are deterministic in the log-probs they output *
- Randomness is introduced by:
  - Prompt variance: multiple ways of prompting the same question, e.g. “Find the square root of N” vs “What multiplied by itself equals N?”
  - Sampling: Using randomness to sample the next token in text generation

* Parallelism and floating point arithmetic can cause non-determinism in some implementations

Demo 0: A Warmup on Randomness

- Open Demo 0 to see how sampling and prompt variance introduce randomness into LLM outputs

Section III: How to Lie with LLM Results

- Goal for this section: understand where LLMs can go wrong when used in research components
- Overview
  - Case study
  - Lie 1: Not considering randomness
  - Lie 2: Cherry picking examples
  - Lie 3: not considering other axes of evaluation
  - Lie 4: Benchmark leakage / data contamination
  - Lie 5: LLM-as-a-judge, rubric drift
  - Lie 6: Human-as-a-judge, inconsistent evaluation
  - Lie 7: Unreported human intervention
  - Lie 8: Not reporting cost / GPU hours
  - Lie 9: Conflating Formatting Abilities with Technical Abilities
  - Hands-on demo

Lie #1: Not Considering Randomness

- LLM outputs are *random*; reporting metrics for a single output can be misleading
- You should report: *What is the probability/likelihood that a model outputs the correct solution?*
- This is true of any system with LLM subcomponents
- Many works/benchmarks employ this through probabilistic metrics e.g. mean accuracy, pass@k
- Often only sampling variance is covered
- Prompt variance matters! (Y. Zi et al. “More Than a Score: Probing the Impact of Prompt Specificity on LLM Code Generation”)

Lie #2: Cherry Picking Examples

- Another facet of random outputs from LLMs is that we must report all results
- Only reporting the “best” results can mislead to thinking the LLM is doing better than it actually is
- Sampling parameters need to be transparent: If the LLM tool only works for some sampling parameters, this is *ok*, but it needs to be reported

Lie #3: Ignoring Important Axes of Evaluation

- To avoid lies 1-2 practitioners often sample many outputs and, thus, need easy-to-automatically-compute metrics for evaluation
- This often leads to Lie #3: using too simple of metrics that don't paint the whole picture
- Consider evaluating LLMs on generating MPI code

This is functionally correct, but not really correct

```
def compute_and_share(data):
    output = compute(data)
    for rank in other_ranks:
        mpi_send(output, rank)
```

This is what we want to see the LLM generate

```
def compute_and_share(data):
    output = compute(data)
    mpi_bcast(output)
```

Lie #4: Benchmark Leakage and Data Contamination

- If your benchmark is in the training set, then the model could be over-fitting the answers
- You're testing how well the model can memorize your problems and not actually solve them
- This is only okay if your benchmark is *exhaustive*, which is almost never the case
- Even if your questions are private, make sure they are not too similar to existing public sources

Lie #5: LLM-as-a-Judge, Rubric Drift

- Rubric drift: small changes in prompts and judge models can drastically alter results
- LLM judging often conflates style and correctness
- LLM-based evaluation is a useful new research tool, but is *insufficient* and *potentially dangerous* for scientific evaluation

Lie #6: Human-as-a-Judge, Inconsistent Eval

- Humans are just as prone to evaluation biases as LLMs
- Evaluation criteria not public or precise. Evaluation data not public
- Social science fields have best practices for using humans to evaluate / rank / label data. It is still very difficult to implement correctly.
- As with lie #5, human judgement is *insufficient* and *dangerous* in scientific results reporting

Lie #7: Unreported Human Intervention

- LLM / Agent pipeline relied on human intervention somewhere and it is not reported
  - Data cleaning between components
  - Prompt design and/or variation
  - Sampling parameter and/or model choice changes
- Approach is not reproducible without documenting and studying these human interventions

Lie #8: Not Reporting Cost / GPU Hours

- API costs or GPU time when self-hosting are imperative to report, particularly when comparing with other approaches
- If an approach yields better results, but costs significantly more this is of interest to users of the approach
- All costs and GPU hours need to be reported
  - Costs fluctuate, so token counts should also be reported.

Lie #9: Conflating Formatting Abilities with Technical Abilities

- If we are testing an LLM/Agent on its ability to do task *X*, then we don't want to conflate performance on *X* with performance on formatting outputs for task *X*.
- An example: You task an LLM with solving a math problem and outputting the final number; your script checks accuracy with `` `int(llm_answer) == ground_truth` ``.
  - The LLM would get this wrong if it generated "The answer is …"
  - Even if it knew the answer we've marked it as wrong
- Give LLMs the benefit of the doubt in parsing outputs, unless your task *X* includes formatting as part of the task

Demo 1: Visualizing Misleading Results

- See the interactive demo that walks through examples for each ”lie”.

Section IV: Experimental Design for Stochastic Components

- Goal for this section: how to design experiments that rely on stochastic subcomponents
- Overview
  - Why careful consideration is important
  - Probabilistic metrics
  - Structured inputs/outputs; mitigating stochasticity
  - Testing individual subcomponents
  - Ablation studies on system
  - Human validation

Section V: Transparency and Reproducibility

- Goal for this section: how to ensure LLM subcomponents are reproducible
- Overview
  - Reproducibility requires transparency
  - What to report
  - Logging LLM inputs/outputs
  - Anonymizing LLM artifacts for peer review

Section VI: Human Validation: When and How

- Goal for this section: how should humans interact with LLM research components
- Overview
  - When to keep humans in the loop
  - How to employ human evaluation on LLM subcomponents

Section VII: Baselines, Metrics, and Ablations

- Goal for this section: best practices for experimental setups for LLM-based research
- Overview
  - What's a good baseline
  - Metrics for LLMs
  - How to structure ablation studies

Section VIII: HPC and Scientific Computing Specifics

- Goal for this section: HPC-specific pitfalls of using LLMs in research
- Overview
  - Parallel code and performance
  - Numerical precision
  - Beyond unit tests
  - Mapping science to code

Conclusion

- Thanks for joining!